Supersetid :8003253

Digital Nurture 5.0 Deep Skilling Handbook Java FSE - Solution

DESIGN PATTERNS AND PRINCIPLES

Supersetid :8003253

Exercise 1: Implementing the Singleton Pattern

```
public class SingletonPatternExample {
   static class Logger {
      private static Logger loggerInstance;
      private Logger() {
         System.out.println("Logger has been initialized.");
      }
      public static Logger getLogger() {
         if (loggerInstance == null) {
            loggerInstance = new Logger();
         }
         return loggerInstance;
      }
      public void writeLog(String message) {
         System.out.println("[LOG] " + message);
      }
   }
   public static void main(String[] args) {

      Logger firstLogger = Logger.getLogger();
      firstLogger.writeLog("Application is starting...");

      Logger secondLogger = Logger.getLogger();
      secondLogger.writeLog("User logged into the system.");

      if (firstLogger == secondLogger) {
         System.out.println("Only one Logger object exists.");
      } else {
         System.out.println("Multiple Logger objects exist.");
      }
   }}
```

OUTPUT:

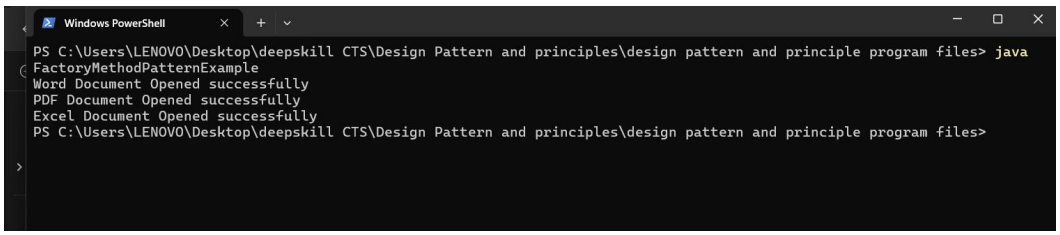
Supersetid :8003253

Exercise 2: Implementing the Factory Method Pattern

```
interface Document {
    void open();
}
class WordDocument implements Document {
    public void open() {
        System.out.println("Word Document Opened successfully");
    }
}
class PdfDocument implements Document {
    public void open() {
        System.out.println("PDF Document Opened successfully");
    }
}
class ExcelDocument implements Document {
    public void open() {
        System.out.println("Excel Document Opened successfully");
    }
}
abstract class DocumentFactory {
    public abstract Document createDocument();
}
class WordFactory extends DocumentFactory {
    public Document createDocument() {
        return new WordDocument();
    }
}
class PdfFactory extends DocumentFactory {
    public Document createDocument() {
        return new PdfDocument();
    }
}
class ExcelFactory extends DocumentFactory {
    public Document createDocument() {
        return new ExcelDocument();
    }
}
public class FactoryMethodPatternExample {

    public static void main(String[] args) {

        DocumentFactory wordFactory = new WordFactory();
```

```
        Document word = wordFactory.createDocument();
        word.open();

        DocumentFactory pdfFactory = new PdfFactory();
        Document pdf = pdfFactory.createDocument();
        pdf.open();

        DocumentFactory excelFactory = new ExcelFactory();
        Document excel = excelFactory.createDocument();
        excel.open();
    }
}
```

OUTPUT:



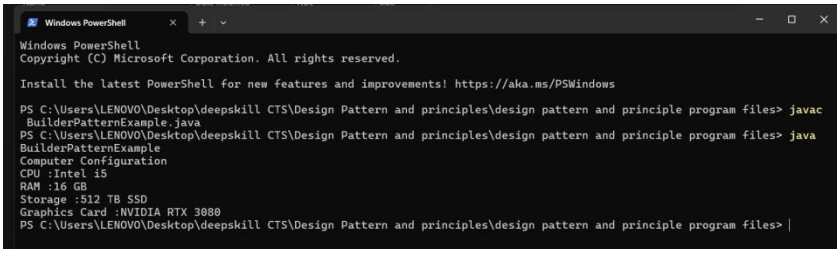
Exercise 3: Implementing the Builder Pattern

```
public class BuilderPatternExample {
    static class Computer {
        private String cpu;
        private String ram;
        private String storage;
        private String graphicsCard;
        private Computer(Builder builder) {
            this.cpu = builder.cpu;
            this.ram = builder.ram;
            this.storage = builder.storage;
            this.graphicsCard = builder.graphicsCard;
        }
        public void showDetails() {
            System.out.println("Computer Configuration");
            System.out.println("CPU :" + cpu);
            System.out.println("RAM :" + ram);
            System.out.println("Storage :" + storage);
            System.out.println("Graphics Card :" + graphicsCard);
        }
        static class Builder {
```

Supersetid :8003253

```
        private String cpu;
        private String ram;
        private String storage;
        private String graphicsCard;
       public Builder setCPU(String cpu) {
            this.cpu = cpu;
            return this;
        }

        public Builder setRAM(String ram) {
            this.ram = ram;
            return this;
        }

        public Builder setStorage(String storage) {
            this.storage = storage;
            return this;
        }

        public Builder setGraphicsCard(String graphicsCard) {
            this.graphicsCard = graphicsCard;
            return this;
        }

        public Computer build() {
            return new Computer(this);
        }
    }
}
public static void main(String[] args) {

    Computer user1 = new Computer.Builder()
            .setCPU("Intel i5")
            .setRAM("16 GB")
            .setStorage("512 TB SSD")
            .setGraphicsCard("NVIDIA RTX 3080")
            .build();
```

OUTPUT:



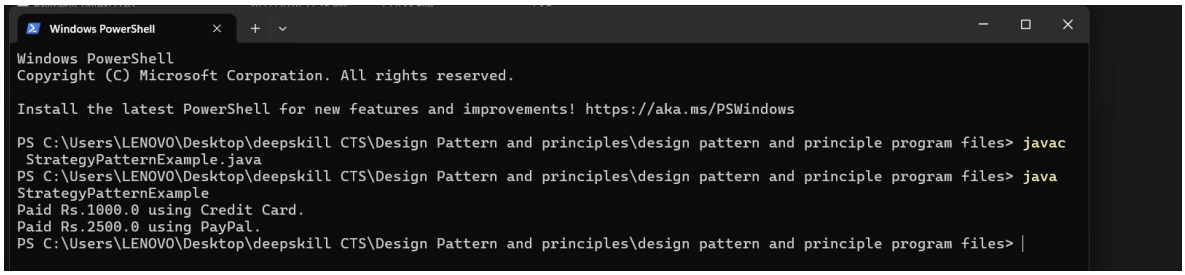
Supersetid :8003253

Exercise 4: Implementing the Adapter Pattern

```
interface PaymentProcessor {
    void processPayment(double amount);
}
class PayPalGateway {

    public void makePayment(double amount) {
        System.out.println("Payment of Rs." + amount + " made using PayPal.");
    }
}
class StripeGateway {

    public void payAmount(double amount) {
        System.out.println("Payment of Rs." + amount + " made using Stripe.");
    }
}
class PayPalAdapter implements PaymentProcessor {

    private PayPalGateway paypal;

    public PayPalAdapter() {
        paypal = new PayPalGateway();
    }
    @Override
    public void processPayment(double amount) {
        paypal.makePayment(amount);
    }
}
class StripeAdapter implements PaymentProcessor {
    private StripeGateway stripe;
    public StripeAdapter() {
        stripe = new StripeGateway();
    }

    @Override
    public void processPayment(double amount) {
        stripe.payAmount(amount);
    }
}
public class AdapterPatternExample {
   public static void main(String[] args) {
        PaymentProcessor payment1 = new PayPalAdapter();
        payment1.processPayment(1000);
        PaymentProcessor payment2 = new StripeAdapter();
```

```
        payment2.processPayment(2500);
    }

}
```

OUTPUT:



Exercise 5: Implementing the Decorator Pattern

```
interface Notifier {
    void send(String message);
}
class EmailNotifier implements Notifier {
    @Override
    public void send(String message) {
        System.out.println("Email Notification: " + message);
    }
}
abstract class NotifierDecorator implements Notifier {
protected Notifier notifier;
public NotifierDecorator(Notifier notifier) {
        this.notifier = notifier;
    }
}
class SMSNotifierDecorator extends NotifierDecorator {
public SMSNotifierDecorator(Notifier notifier) {
        super(notifier);
    }
    @Override
    public void send(String message) {
        notifier.send(message);
        System.out.println("SMS Notification: " + message);
    }
}
class SlackNotifierDecorator extends NotifierDecorator {
```

Supersetid :8003253

```
    public SlackNotifierDecorator(Notifier notifier) {
        super(notifier);
    }
    @Override
    public void send(String message) {
        notifier.send(message);
        System.out.println("Slack Notification: " + message);
    }
}
public class DecoratorPatternExample {

    public static void main(String[] args) {
        Notifier email = new EmailNotifier();
        email.send("Server Started");
        System.out.println();
        Notifier emailSMS = new SMSNotifierDecorator(new EmailNotifier());
        emailSMS.send("Payment Successful");
        System.out.println();
        Notifier allNotifications =
                new SlackNotifierDecorator(
                new SMSNotifierDecorator( new EmailNotifier()));
        allNotifications.send("System Error");
    }}
```

OUTPUT:

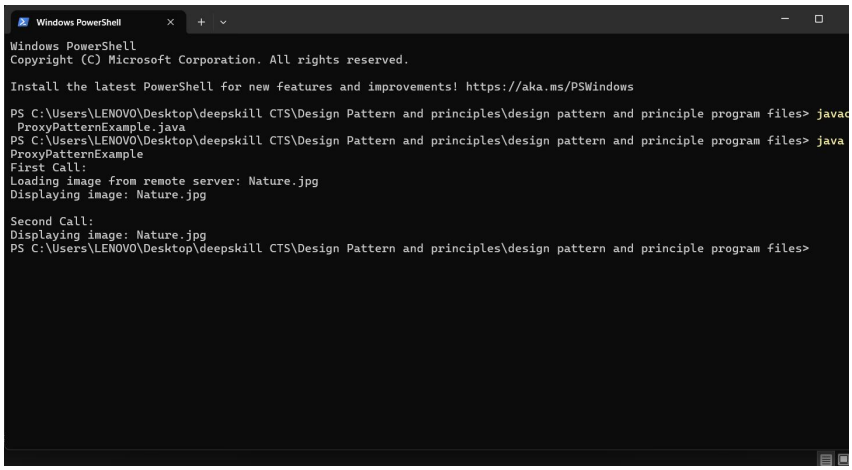
Supersetid :8003253

Exercise 6: Implementing the Proxy Pattern

```
interface Image {
    void display();
}
class RealImage implements Image {

    private String fileName;
    public RealImage(String fileName) {
        this.fileName = fileName;
        loadImage();
    }
    private void loadImage() {
        System.out.println("Loading image from remote server: " + fileName);
    }
    @Override
    public void display() {
        System.out.println("Displaying image: " + fileName);
    }
}
class ProxyImage implements Image {
    private String fileName;
    private RealImage realImage;
    public ProxyImage(String fileName) {
        this.fileName = fileName;
    }
    @Override
    public void display() {
        if (realImage == null) {
            realImage = new RealImage(fileName);
        }
        realImage.display();
    }
}
public class ProxyPatternExample {
    public static void main(String[] args) {
        Image image = new ProxyImage("Nature.jpg");
        System.out.println("First Call:");
        image.display();
        System.out.println();
        System.out.println("Second Call:");
        image.display();
    }

}
```

Supersetid :8003253

OUTPUT:



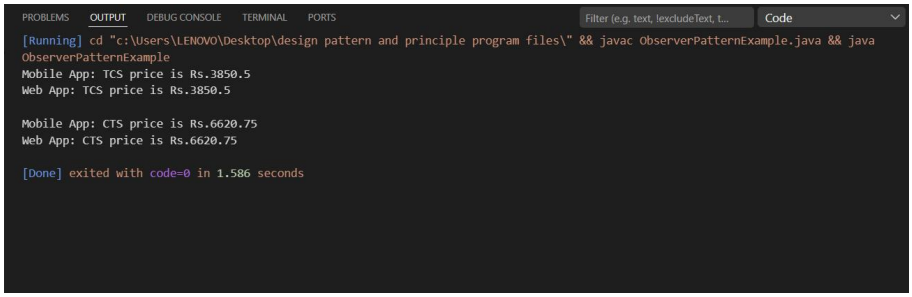
Exercise 7: Implementing the Observer Pattern

```
import java.util.ArrayList;
import java.util.List;
interface Observer {
    void update(String stockName, double price);
}
interface Stock {
    void registerObserver(Observer observer);
    void removeObserver(Observer observer);
    void notifyObservers();
}
class StockMarket implements Stock {
    private List<Observer> observers = new ArrayList<>();
    private String stockName;
    private double price;
    @Override
    public void registerObserver(Observer observer) {
        observers.add(observer);
    }
    @Override
    public void removeObserver(Observer observer) {
        observers.remove(observer);
    }
    @Override
    public void notifyObservers() {
        for (Observer observer : observers) {
            observer.update(stockName, price);
        }
```

```
    }
    public void setStock(String stockName, double price) {
        this.stockName = stockName;
        this.price = price;
        notifyObservers();
    }
}
class MobileApp implements Observer {

    @Override
    public void update(String stockName, double price) {
        System.out.println("Mobile App: " + stockName + " price is Rs." + price);
    }
}
class WebApp implements Observer {

    @Override
    public void update(String stockName, double price) {
        System.out.println("Web App: " + stockName + " price is Rs." + price);
    }
}
public class ObserverPatternExample {

    public static void main(String[] args) {

        StockMarket stockMarket = new StockMarket();
        Observer mobile = new MobileApp();
        Observer web = new WebApp();
        stockMarket.registerObserver(mobile);
        stockMarket.registerObserver(web);
        stockMarket.setStock("TCS", 3850.50);
        System.out.println();
        stockMarket.setStock("CTS", 6620.75);
    }
}
```

Supersetid :8003253

OUTPUT:



Exercise 8: Implementing the Strategy Pattern

```
interface PaymentStrategy {
    void pay(double amount);
}
class CreditCardPayment implements PaymentStrategy {
    @Override
    public void pay(double amount) {
        System.out.println("Paid Rs." + amount + " using Credit Card.");
    }
}
class PayPalPayment implements PaymentStrategy {
    @Override
    public void pay(double amount) {
        System.out.println("Paid Rs." + amount + " using PayPal.");
    }
}
class PaymentContext {
    private PaymentStrategy paymentStrategy;
    public void setPaymentStrategy(PaymentStrategy paymentStrategy) {
        this.paymentStrategy = paymentStrategy;
    }
    public void makePayment(double amount) {
        paymentStrategy.pay(amount);
    }
}
public class StrategyPatternExample {

    public static void main(String[] args) {

        PaymentContext payment = new PaymentContext();
        payment.setPaymentStrategy(new CreditCardPayment());
        payment.makePayment(1000);
        payment.setPaymentStrategy(new PayPalPayment());
```

```
        payment.makePayment(2500);
    }
}
```

OUTPUT:

Exercise 9: Implementing the Command Pattern

```
interface Command {
    void execute();
}
class Light {

    public void turnOn() {
        System.out.println("Light is ON");
    }

    public void turnOff() {
        System.out.println("Light is OFF");
    }
}
class LightOnCommand implements Command {

    private Light light;

    public LightOnCommand(Light light) {
        this.light = light;
    }

    @Override
    public void execute() {
        light.turnOn();
    }
}
class LightOffCommand implements Command {
```

```
    private Light light;

    public LightOffCommand(Light light) {
        this.light = light;
    }

    @Override
    public void execute() {
        light.turnOff();
    }
}
class RemoteControl {

    private Command command;

    public void setCommand(Command command) {
        this.command = command;
    }

    public void pressButton() {
        command.execute();
    }
}
public class CommandPatternExample {

    public static void main(String[] args) {

        Light light = new Light();
        Command lightOn = new LightOnCommand(light);
        Command lightOff = new LightOffCommand(light);
        RemoteControl remote = new RemoteControl();
        remote.setCommand(lightOn);
        remote.pressButton();
        remote.setCommand(lightOff);
        remote.pressButton();
    }
}
```

Supersetid :8003253

OUTPUT:

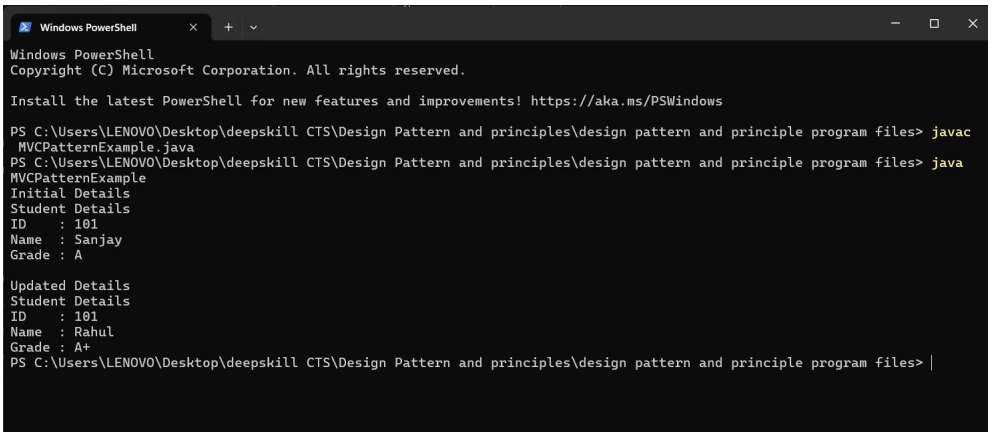
Exercise 10: Implementing the MVC Pattern

```
class Student {

    private int id;
    private String name;
    private String grade;
 public Student(int id, String name, String grade) {
        this.id = id;
        this.name = name;
        this.grade = grade;
    }
   public int getId() {
        return id;
    }

    public void setId(int id) {
        this.id = id;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public String getGrade() {
        return grade;
    }

    public void setGrade(String grade) {
        this.grade = grade;
    }
```

```
}
class StudentView {

    public void displayStudentDetails(Student student) {

        System.out.println("Student Details");
        System.out.println("ID    : " + student.getId());
        System.out.println("Name  : " + student.getName());
        System.out.println("Grade : " + student.getGrade());
    }
}
class StudentController {

    private Student model;
    private StudentView view;

    public StudentController(Student model, StudentView view) {
        this.model = model;
        this.view = view;
    }

    public void setStudentName(String name) {
        model.setName(name);
    }

    public void setStudentGrade(String grade) {
        model.setGrade(grade);
    }

    public void updateView() {
        view.displayStudentDetails(model);
    }
}
public class MVCPatternExample {

    public static void main(String[] args) {

        Student student = new Student(101, "Sanjay", "A");
        StudentView view = new StudentView();
        StudentController controller =
                new StudentController(student, view);
        System.out.println("Initial Details");
        controller.updateView();
        System.out.println();
        controller.setStudentName("Rahul");
```

```
        controller.setStudentGrade("A+");
        System.out.println("Updated Details");
        controller.updateView();
    }
}
```

OUTPUT:



Exercise 11: Implementing Dependency Injection

```
interface CustomerRepository {
    void findCustomerById(int id);
}
class CustomerRepositoryImpl implements CustomerRepository {
    @Override
    public void findCustomerById(int id) {
        System.out.println("Customer found with ID : " + id);
    }
}
class CustomerService {
    private CustomerRepository repository;

    public CustomerService(CustomerRepository repository) {
        this.repository = repository;
    }
    public void getCustomer(int id) {
        repository.findCustomerById(id);
    }
}
public class DependencyInjectionExample {

    public static void main(String[] args) {
```

Supersetid :8003253

```
        CustomerRepository repository = new CustomerRepositoryImpl();
        CustomerService service = new CustomerService(repository);
        service.getCustomer(101);
    }
}
```

OUTPUT: